

AWS 3-TIER JAVA APPLICATION

Deployment Guide

Spring Boot · Apache Tomcat · Amazon RDS (MySQL) · AWS Console

Project prefix: javaauth- · Region: us-east-1

Prepared by

Muhammad Abdullah Muzaffar

DevOps & Cloud Engineer

Includes a full incident log of the real errors hit during this build and how each was resolved.

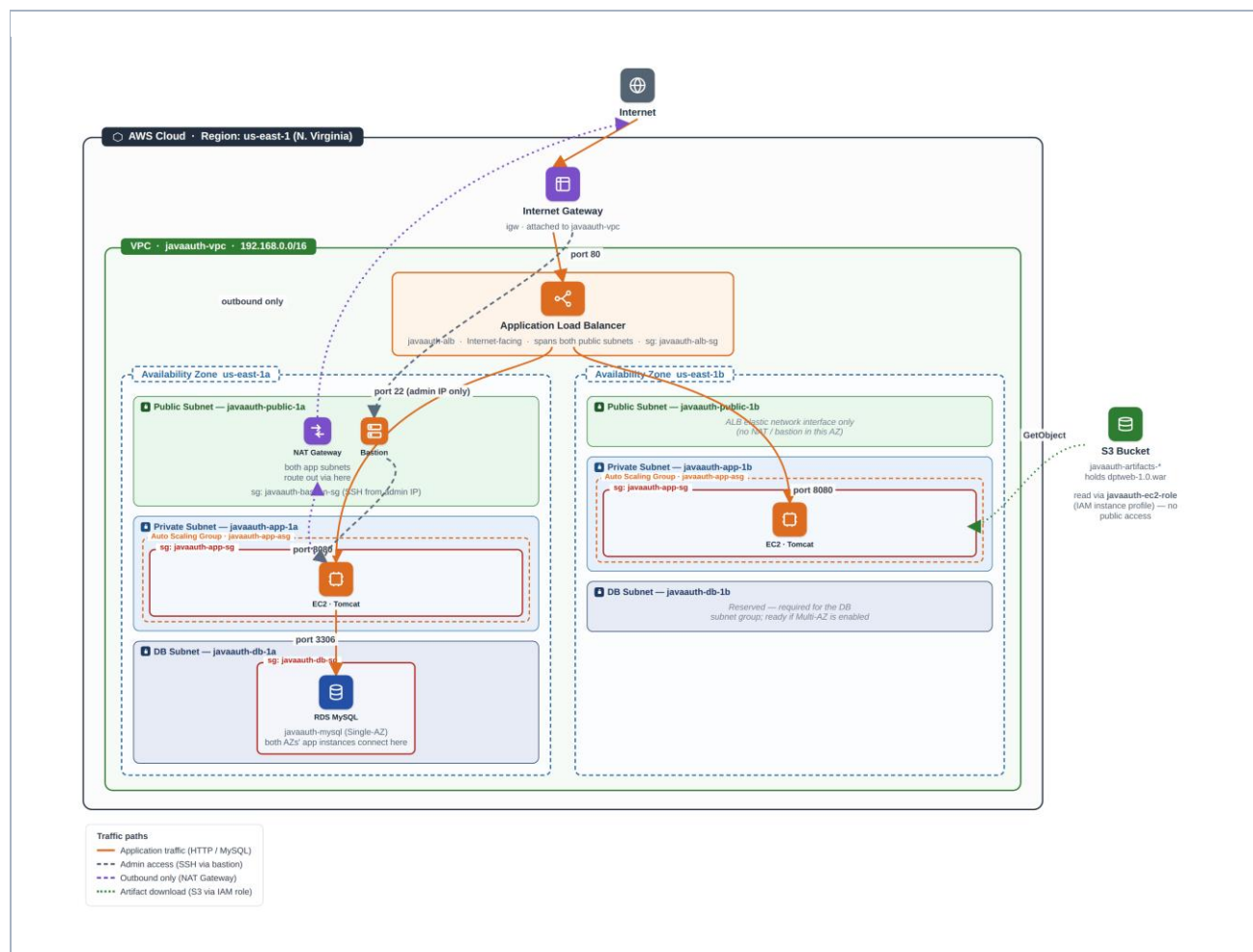
Project Overview

This project takes a Java web application from a plain login form on a laptop to a real, internet-facing deployment on AWS — built by hand in the Console, one AWS service at a time, so every piece of the architecture is understood rather than hidden behind a single automated script.

The design is a classic three-tier split, and the whole point of it is isolation: a visitor's browser only ever talks to a public Load Balancer. The Load Balancer forwards that request to a Java (Spring Boot) application running on Tomcat — but those servers sit in a private subnet with no direct route in from the internet at all. That application, in turn, is the only thing allowed to reach the MySQL database, which is private end-to-end. The app servers run in an Auto Scaling Group spread across two Availability Zones, so the site survives a single server — or a whole zone — going down.

What this build uses:

- Compute — Amazon EC2 (Amazon Linux 2023, Java 11, Apache Tomcat 9) in an Auto Scaling Group
- Database — Amazon RDS for MySQL, in its own private subnet, never internet-reachable
- Networking — a custom VPC across 2 Availability Zones: public, private-app, and private-database subnets, an Internet Gateway, and a NAT Gateway
- Load balancing — an internet-facing Application Load Balancer in front of the app tier
- Storage — Amazon S3 holds the built application artifact, read at boot via an IAM role (no public access)
- Admin access — a small bastion host, since nothing else in the VPC is reachable from the internet



Complete infrastructure — the javaauth 3-tier build

Solid = application traffic · dashed grey = admin SSH · dashed purple = outbound-only · dotted green = artifact download

Everything from here on follows the exact order this was actually built in the AWS Console: network first, then the database, then the compute layer, then the public-facing pieces, then automation and monitoring.

About This Guide

A few places in the walkthrough hit a real, live error during the build. Those points are flagged inline with a highlighted box like the one below, so you can see exactly where in the sequence things went wrong — without breaking up the main steps with lengthy troubleshooting text.

Real issue hit at this step — Troubleshooting Log #N

Short pointer only. Full symptom, root cause, and fix are in the Troubleshooting & Error Resolution Log at the end of this document.

Ground rules used throughout the build:

- Region: N. Virginia (us-east-1) for every resource, start to finish.
- No root account for daily work — an IAM admin user was used instead.
- Every resource is named with the javaauth- prefix, to make cleanup and auditing simple.
- A private database subnet is separate from the private app subnet — RDS and the app tier never share a subnet.

Estimated Monthly Cost

Resource	Approximate cost
1 × NAT Gateway	~\$32/month + data processed
Application Load Balancer	~\$16/month
RDS db.t3.micro (Single-AZ)	Free-tier eligible, or ~\$13/month
2 × t3.micro EC2 (app tier)	Free-tier eligible, or ~\$8/month each
Elastic IP (if left unattached)	Billed hourly if unused

HTTPS, Multi-AZ RDS, CloudFront, ElastiCache and a second VPC are all deliberately left out of the base build below — see “Recommended Next Steps” near the end. Added together they can push the monthly cost past \$100.

PHASE 0 Local Workspace Preparation

Before touching the AWS Console, the local machine needs a JDK and Maven so the application can be compiled into a deployable WAR file later in Phase 5.

JDK	Version 11 — Eclipse Temurin or Amazon Corretto
Build tool	Apache Maven, added to the system PATH
Verify	java -version mvn -version



Figure 1 — Local machine — installing the JDK and Maven

winget install EclipseAdoptium.Temurin.11.JDK · winget install Apache.Maven

The WAR is not built yet at this point — the database has to exist first so its endpoint can go into the application's configuration. Building happens in Phase 5, after Phase 3 (RDS).

PHASE 1 VPC and Networking

Everything else in this build sits on top of these subnets, so getting this phase right matters more than any other single step.

1.1 Create the VPC with the wizard

AWS Console → VPC → Create VPC → “VPC and more” (not “VPC only”).

Name tag auto-generation	javaauth
IPv4 CIDR	192.168.0.0/16
IPv6	No
Number of AZs	2
Public subnets	2
Private subnets	2
NAT gateways	In 1 AZ
VPC endpoints	None
DNS hostnames / resolution	Enabled

This single wizard run creates the VPC, two public subnets, two private subnets, an Internet Gateway, one NAT Gateway, and the route tables to go with them.

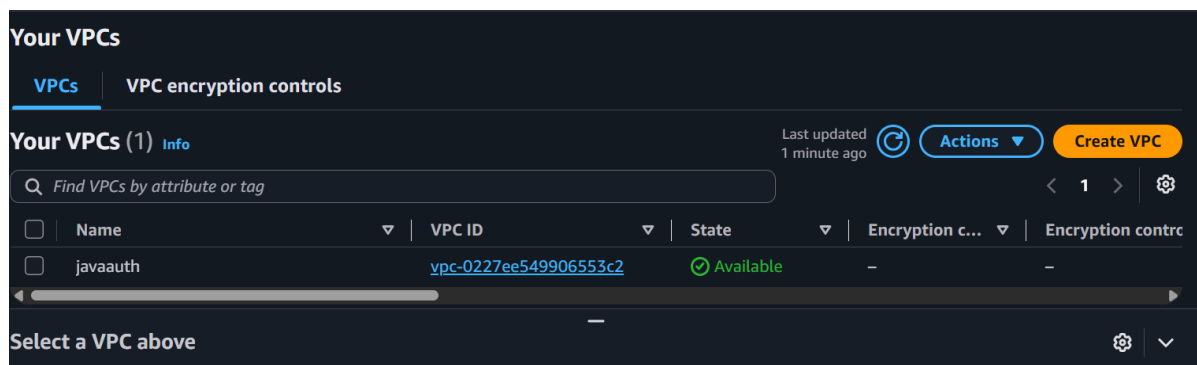


Figure 2 — VPC console — javaauth-vpc created
 Your VPCs → javaauth · vpc-0227ee549906553c2 · State: Available

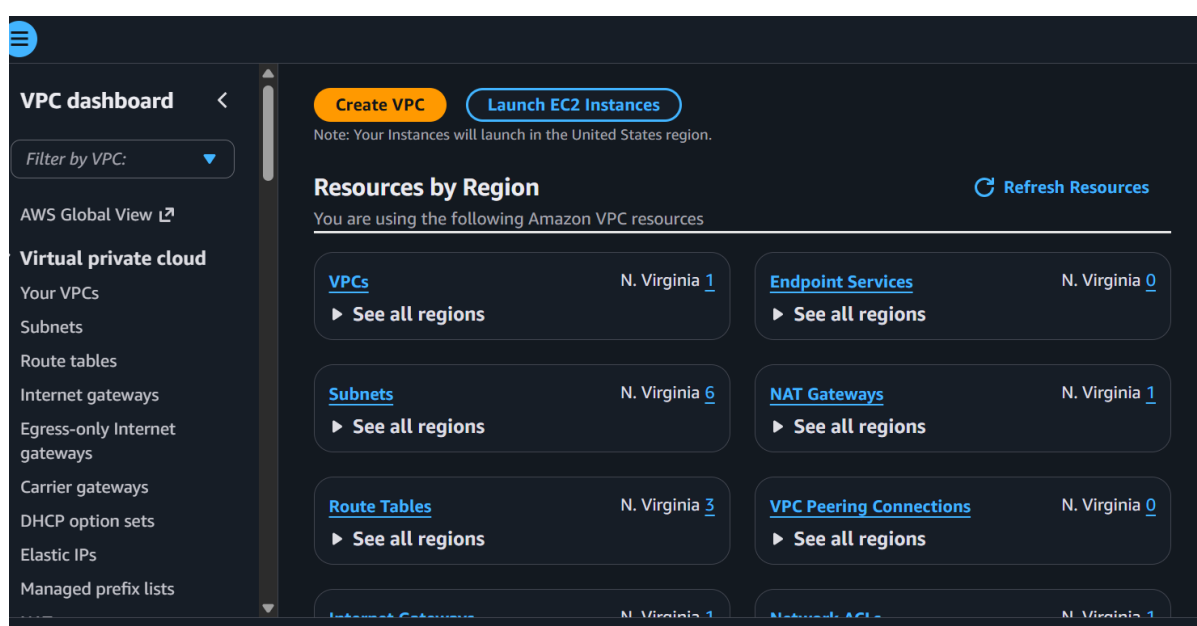


Figure 3 — VPC console — everything the wizard created
 1 VPC · 6 subnets · 3 route tables · 1 Internet Gateway · 1 NAT Gateway, all in N. Virginia

1.2 Rename the subnets

Original	Renamed to
Public — us-east-1a	javaauth-public-1a
Public — us-east-1b	javaauth-public-1b
Private — us-east-1a	javaauth-app-1a
Private — us-east-1b	javaauth-app-1b

1.3 Add two dedicated database subnets

The app tier and RDS should never share a subnet, so two more private subnets are added specifically for the database.

Subnet	AZ	CIDR
javaauth-db-1a	us-east-1a	192.168.21.0/24
javaauth-db-1b	us-east-1b	192.168.22.0/24

VPC → Subnets → Create subnet, both under javaauth-vpc. Associate each new DB subnet with the private route table — the one whose default route (0.0.0.0/0) points at the NAT Gateway, not the Internet Gateway.

Name	Subnet ID	State	VPC
javaauth-app-1a	subnet-009fa87ce1f6e9411	Available	vpc-0227ee549906553c2
javaauth-public-1b	subnet-01f83603768ecdbf1	Available	vpc-0227ee549906553c2
javaauth-public-1a	subnet-0338507b1e079d115	Available	vpc-0227ee549906553c2
javaauth-db-1a	subnet-0085425abc728f88f	Available	vpc-0227ee549906553c2

Figure 4 — VPC console — all six subnets
Public, app, and DB subnets renamed and created under javaauth-vpc

Route table reference

Tier	Default route	Purpose
Public subnet	0.0.0.0/0 → Internet Gateway	ALB and the bastion live here
Private app subnet	0.0.0.0/0 → NAT Gateway	App can reach package repos / AWS APIs; the internet cannot reach the instance directly
DB subnet	No direct IGW route	RDS stays fully private; it does not need the NAT Gateway either

Name	Route table ID	Explicit subnet associ...	Edge associations	Main
default-route	rtb-05f646f0eca8f0981	—	—	Yes
public-rt	rtb-043113bc83d0378b0	2 subnets	—	No
private-rt	rtb-0b451493ed5d214d9	4 subnets	—	No

Figure 5 — VPC console — route tables
public-rt (2 subnets) and private-rt (4 subnets), alongside the VPC's default table

Name	Internet gateway ID	State	VPC ID
internet-gateway	igw-026681d0d40e2df9a	Attached	vpc-0227ee549906553c2

Figure 6 — VPC console — Internet Gateway
internet-gateway · igw-026681d0d40e2df9a · Attached to javaauth-vpc

⚠ Real issue hit at this step — Troubleshooting Log #4

The NAT Gateway must be created in a public subnet (javaauth-public-1a). Placing it in a private subnet by mistake is what caused the app tier's internet timeout later in Phase 7 — full story in the log.

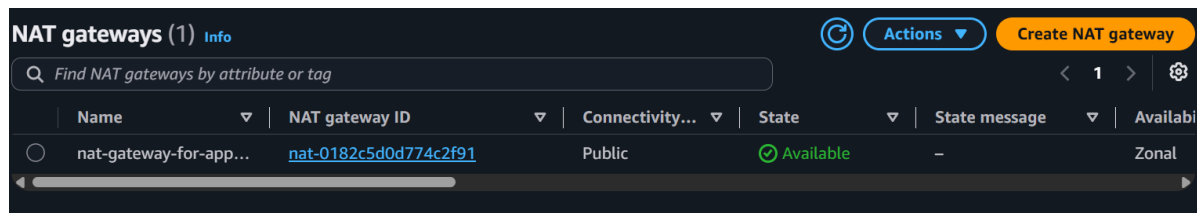


Figure 7 — VPC console — NAT Gateway, corrected
nat-0182c5d0d774c2f91 · Connectivity: Public · State: Available — now sitting in the public subnet

PHASE 2 Security Groups

Four security groups, each only trusting the specific group in front of it — never a raw IP range for anything except the ALB's public HTTP listener and (temporarily) SSH to the bastion.

Security group	Inbound rule	Source
javaauth-alb-sg	HTTP · TCP 80	0.0.0.0/0
javaauth-alb-sg	HTTPS · TCP 443 (optional)	0.0.0.0/0
javaauth-bastion-sg	SSH · TCP 22	My IP
javaauth-app-sg	Custom TCP 8080	javaauth-alb-sg
javaauth-app-sg	SSH · TCP 22	javaauth-bastion-sg
javaauth-db-sg	MySQL/Aurora · TCP 3306	javaauth-app-sg
javaauth-db-sg	MySQL/Aurora · TCP 3306	javaauth-bastion-sg

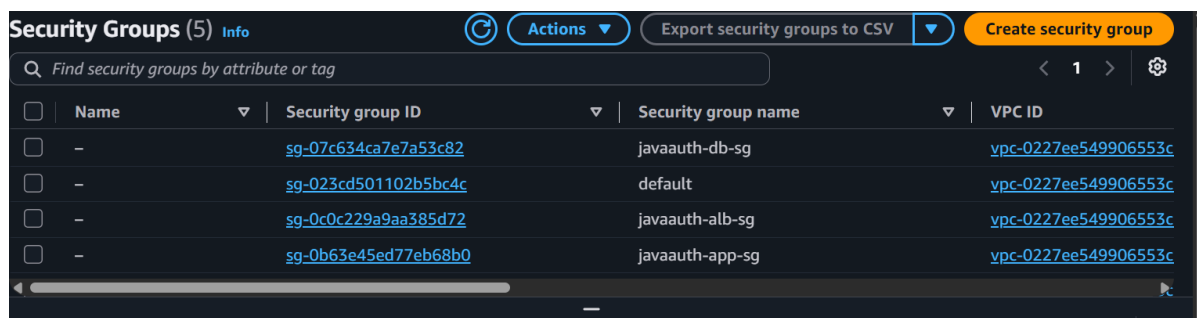


Figure 8 — EC2 console — all four security groups
javaauth-db-sg, javaauth-alb-sg, javaauth-app-sg, and the VPC's default group

Never open port 22 to 0.0.0.0/0. The point of this layout is that traffic is only ever allowed security-group-to-security-group — the database is never reachable from the open internet.

PHASE 3 Database — Amazon RDS (MySQL)

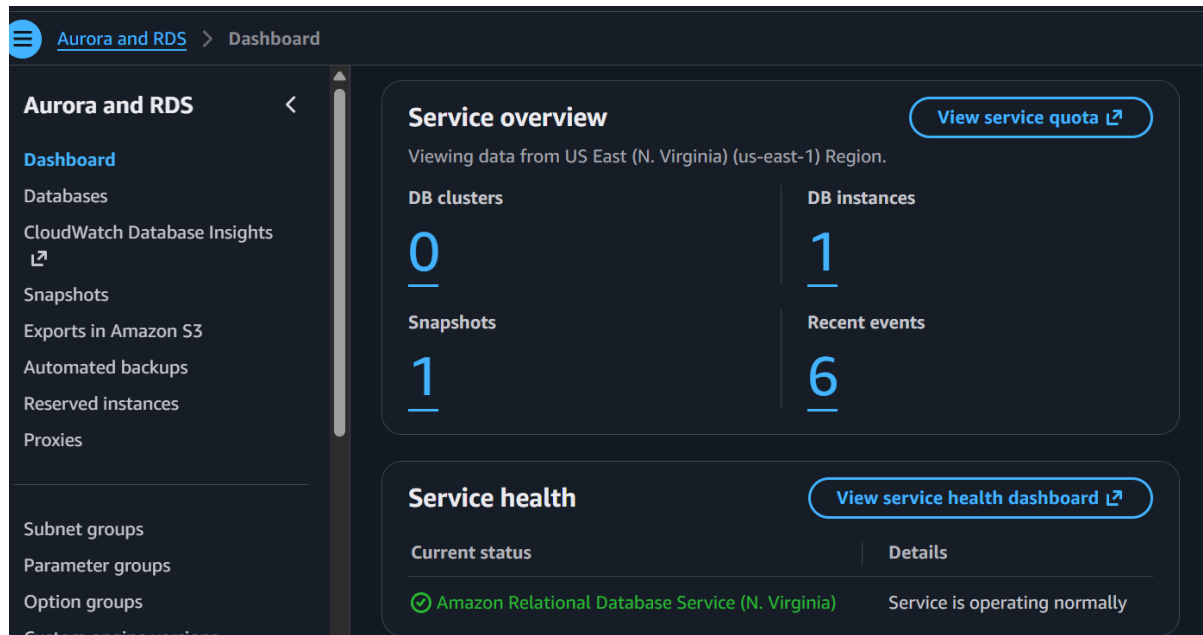


Figure 9 — RDS console — dashboard after the database was created
1 DB instance, N. Virginia region, service status Operating normally

3.1 DB subnet group

RDS → Subnet groups → Create DB subnet group.

Name	javaauth-db-subnets
VPC	javaauth-vpc
Subnets	javaauth-db-1a and javaauth-db-1b

RDS needs subnets in at least two Availability Zones, even for a Single-AZ database instance.

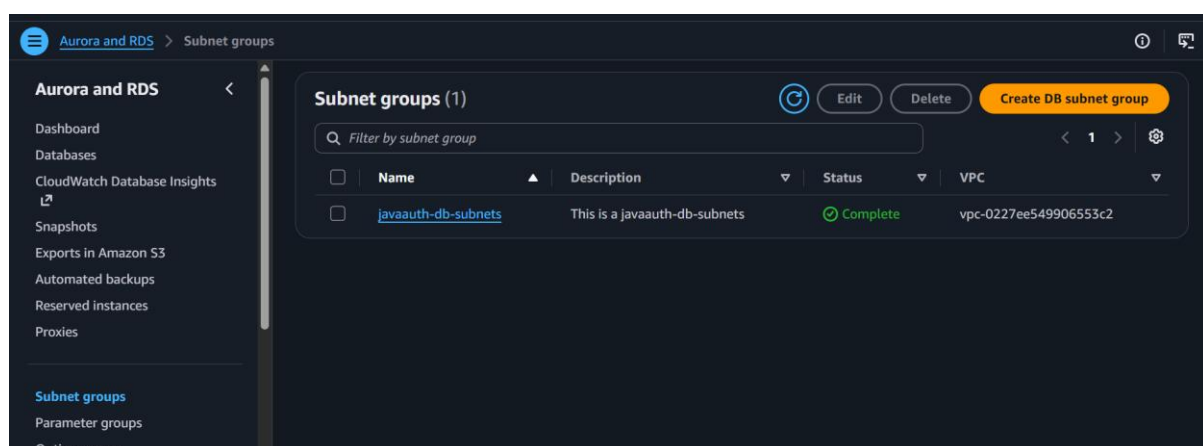


Figure 10 — RDS console — DB subnet group
javaauth-db-subnets · Status: Complete · javaauth-vpc

3.2 Create the RDS instance

Creation method	Standard create
-----------------	-----------------

Engine / version	MySQL 8.0.x
Template	Free tier (if available) or Dev/Test
DB instance identifier	javaauth-mysql
Master username	admin
Master password	Strong password, saved in a password manager
Instance class	Burstable → db.t3.micro
Storage	gp3, 20 GiB, storage autoscaling off
Multi-AZ	No (for this base build)
VPC / subnet group	javaauth-vpc / javaauth-db-subnets
Public access	No
VPC security group	javaauth-db-sg only — remove Default
Initial database name	UserDB
Backup retention	1 day
Deletion protection	Off, for a lab environment

Create the database and wait for Status = Available (usually 5–15 minutes). Open the instance and copy the Endpoint — it looks like javaauth-mysql.xxxxxxxx.us-east-1.rds.amazonaws.com. This value is needed again in Phase 4 and Phase 5.

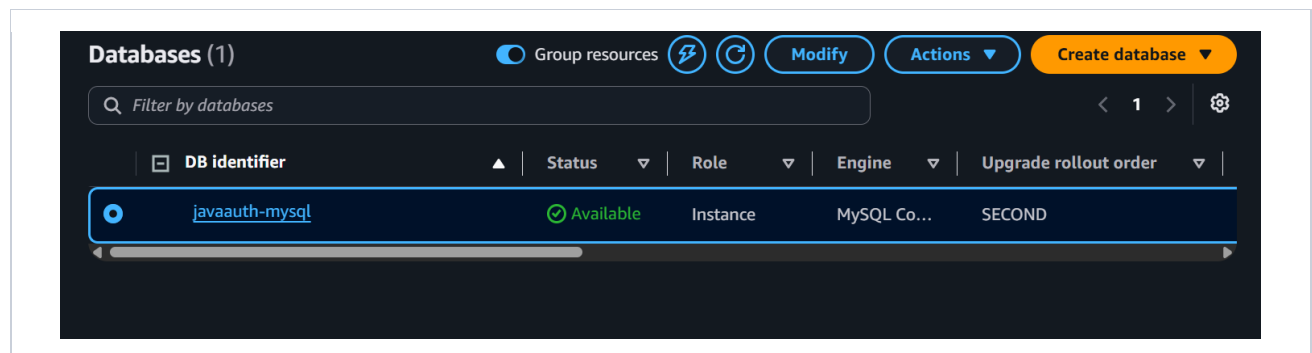


Figure 11 — RDS console — javaauth-mysql instance

Status: Available · Engine: MySQL · Role: Instance

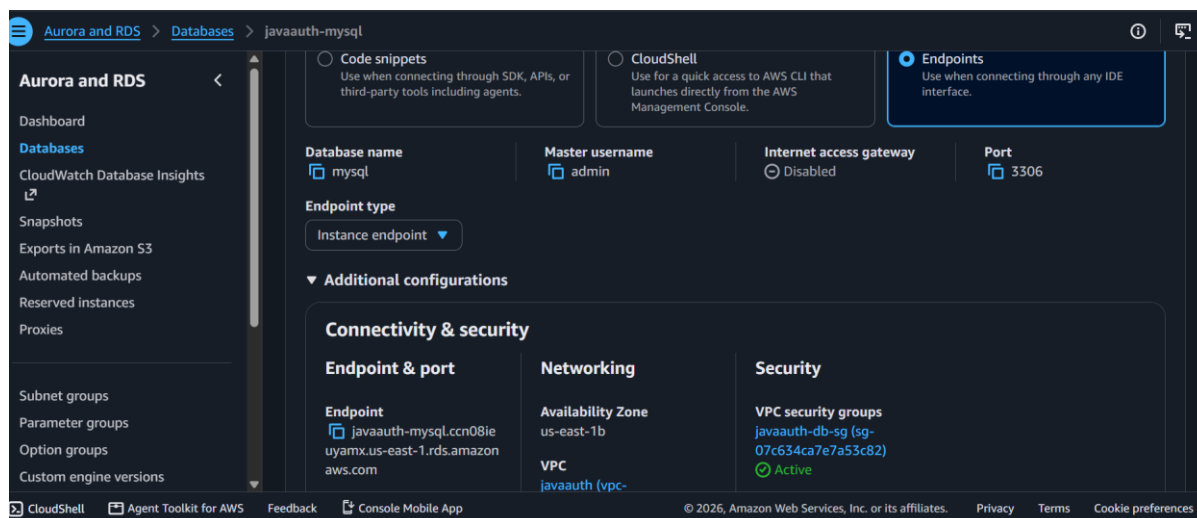


Figure 12 — RDS console — connectivity details

Endpoint javaauth-mysql.ccn08ieuyamx.us-east-1.rds.amazonaws.com : 3306 · security group javaauth-db-sg

PHASE 4 Bastion Host

The RDS instance is private and unreachable directly from a laptop, so a small public EC2 jump host is used to reach it — and later, to reach the private app server too.

4.1 Key pair

Name	javaauth-key
Type / format	RSA · .pem

Download it immediately — AWS will not let you download the private key a second time.

4.2 Launch the bastion

Name	javaauth-bastion
AMI	Amazon Linux 2023
Instance type	t3.micro
Key pair	javaauth-key
Subnet	javaauth-public-1a
Auto-assign public IP	Enable
Security group	javaauth-bastion-sg
Storage	8 GiB gp3

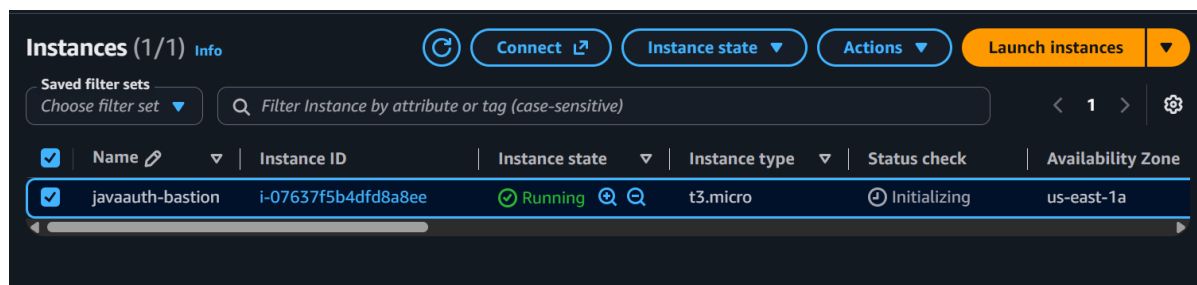


Figure 13 — EC2 console — bastion host launched

javaauth-bastion · i-07637f5b4dfd8a8ee · State: Running · us-east-1a

4.3 Connect and prepare the database

From a local terminal:

```
ssh -i "javaauth-key.pem" ec2-user@BASTION_PUBLIC_IP
```

If Windows complains about key file permissions:

```
icacls "javaauth-key.pem" /inheritance:r
icacls "javaauth-key.pem" /grant:r "$($env:USERNAME):(R)"
```

On the bastion, install the MySQL client and create the schema:

```
sudo dnf install mariadb105 -y
mysql -h RDS_ENDPOINT -u admin -p

SHOW DATABASES;
USE UserDB;
CREATE TABLE Employee (
  id int unsigned auto_increment not null,
  first_name varchar(250),
  last_name varchar(250),
  email varchar(250),
  username varchar(250),
  password varchar(250),
  regdate timestamp,
  primary key (id)
);
DESCRIBE Employee;
EXIT;
```

If the connection fails, double-check the security group, subnet, and RDS endpoint before going further.

PHASE 5 Application Config, Build, and S3 Upload

5.1 Point the app at RDS

Edit Java-Login-App/src/main/resources/application.properties:

```
spring.mvc.view.prefix=/pages/
spring.mvc.view.suffix=.jsp
spring.datasource.url=jdbc:mysql://RDS_ENDPOINT:3306/UserDB
spring.datasource.username=admin
spring.datasource.password=YOUR_RDS_PASSWORD
```

5.2 Build the WAR

```
cd Java-Login-App
```

```
mvn clean package -DskipTests
```

Expected output: Java-Login-App/target/dptweb-1.0.war

⚠ Real issue hit at this step — Troubleshooting Log #1

The very first build attempt failed with “No such host is known (repo.maven.apache.org)” — a local network/DNS problem, not a project problem.

⚠ Real issue hit at this step — Troubleshooting Log #2

The next attempt failed on the MySQL driver dependency's missing version. See the log for the exact pom.xml fix.

Recommended: in pom.xml, mark the tomcat-jasper dependency as provided (see Troubleshooting Log #7) — it prevents a library-conflict crash that otherwise only shows up later, during Phase 7 verification.

5.3 Upload the WAR to S3

Bucket name	javaauth-artifacts-YOURNAME (must be globally unique)
Region	us-east-1
Block all public access	ON

Create the bucket, then upload dptweb-1.0.war. The bucket stays fully private — EC2 will use an IAM role to read from it, not a public URL.

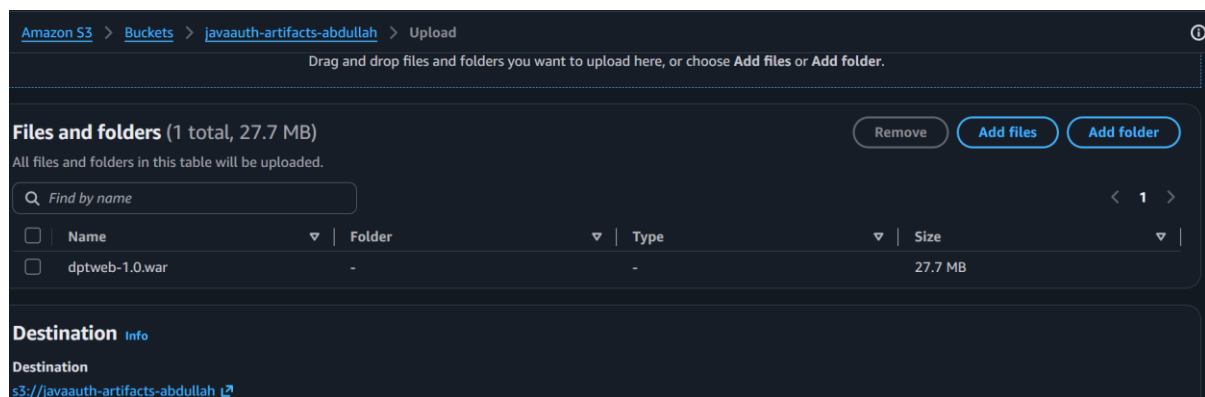


Figure 14 — S3 console — uploading the WAR file
dptweb-1.0.war (27.7 MB) → s3://javaauth-artifacts-abdullah

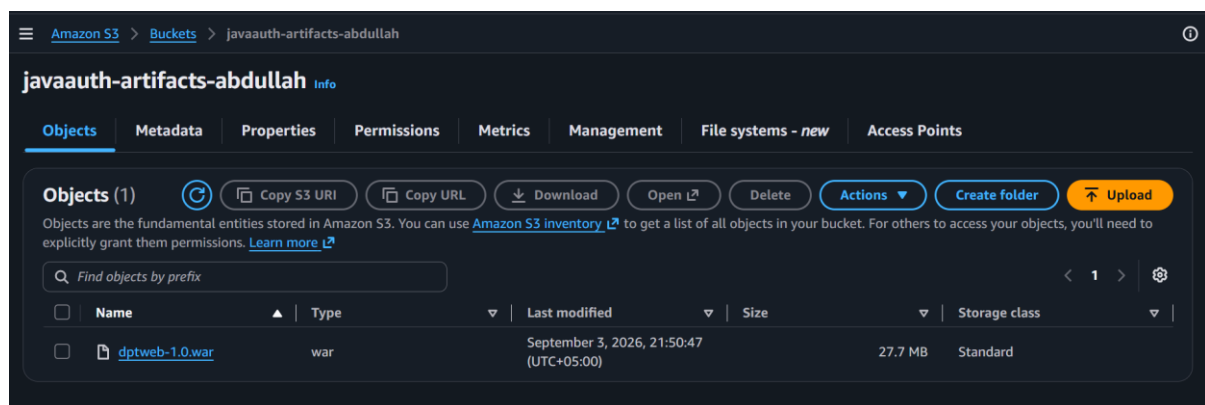


Figure 15 — S3 console — upload confirmed
javaauth-artifacts-abdullah → dptweb-1.0.war, 27.7 MB, Standard storage

PHASE 6 IAM Role for the App Servers

Private-subnet EC2 instances need permission to read from S3 (and, optionally, to publish CloudWatch metrics).

Trusted entity	AWS service → EC2
Policy	AmazonS3ReadOnlyAccess (lab shortcut)
Optional policy	CloudWatchAgentServerPolicy
Role name	javaauth-ec2-role

For a production setup, scope S3 permissions down to just the specific artifact bucket instead of account-wide read-only access.

⚠ Real issue hit at this step — Troubleshooting Log #3

IAM → Roles → Create role failed here with a console “Deserialization error” that had nothing to do with the role configuration itself.

PHASE 7 First Private Tomcat App Server

One Tomcat server is launched manually first. Once it works, Phase 9 automates the same configuration with Auto Scaling.

Name	javaauth-app-1
AMI	Amazon Linux 2023
Instance type	t3.micro
Key pair	javaauth-key
Subnet	javaauth-app-1a (private)
Auto-assign public IP	Disable
Security group	javaauth-app-sg
IAM instance profile	javaauth-ec2-role

7.1 User data (Advanced details)

Paste the following at launch, with the real bucket name from Phase 5.3 in place of YOURNAME:

```
#!/bin/bash
set -e
dnf update -y
dnf install -y java-11-amazon-corretto wget tar
useradd -m -s /bin/bash tomcat || true
cd /opt
wget -q https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.98/bin/apache-tomcat-9.0.98.tar.gz
tar xzf apache-tomcat-9.0.98.tar.gz
mv apache-tomcat-9.0.98 tomcat
chown -R tomcat:tomcat /opt/tomcat
chmod +x /opt/tomcat/bin/*.sh

# WAR from S3
dnf install -y awscli
aws s3 cp s3://javaauth-artifacts-YOURNAME/dptweb-1.0.war /opt/tomcat/webapps/dptweb.war
```

```

chown tomcat:tomcat /opt/tomcat/webapps/dptweb.war

cat >/etc/systemd/system/tomcat.service <<'EOF'
[Unit]
Description=Apache Tomcat
After=network.target
[Service]
Type=forking
User=tomcat
Group=tomcat
Environment=JAVA_HOME=/usr/lib/jvm/java-11-amazon-corretto
Environment=CATALINA_HOME=/opt/tomcat
Environment=CATALINA_BASE=/opt/tomcat
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
Restart=on-failure
[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now tomcat

```

⚠ Real issue hit at this step — Troubleshooting Log #5

A single stray space in the “aws s3 cp” line (“s3:// bucketname” instead of “s3://bucketname”) silently killed this exact script under set -e. The version above is already corrected.

Launch the instance and wait 3–5 minutes for the script to finish.

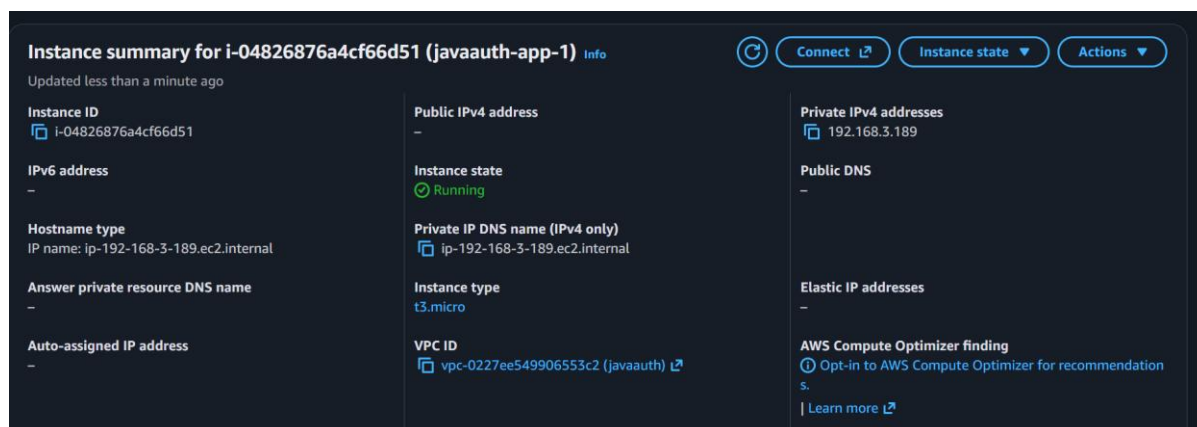


Figure 16 — EC2 console — app server launched

javaauth-app-1 · private IP 192.168.3.189 · t3.micro · State: Running

7.2 SSH to the private app server via the bastion

```
ssh -i "javaauth-key.pem" -J ec2-user@BASTION_PUBLIC_IP ec2-user@APP_PRIVATE_IP
```

⚠ Real issue hit at this step — Troubleshooting Log #6

This exact ProxyJump command repeatedly returned “Permission denied (publickey)” from Windows PowerShell, even with a verified-correct key. The workaround that actually worked is in the log.

On the app server:

```

sudo systemctl status tomcat
curl -I http://localhost:8080/dptweb/

```

A healthy result is HTTP 200 (or a real JSP response) — not “connection refused”.

⚠ Real issue hit at this step — Troubleshooting Log #7

The status came back active (running), but curl returned HTTP 404. The real cause was a library conflict inside the WAR file, not a Tomcat problem — full detail and the permanent pom.xml fix are in the log.

```
[ec2-user@ip-192-168-3-123 ~]$ curl -I http://localhost:8080/dptweb/login
HTTP/1.1 401
WWW-Authenticate: Basic realm="Realm"
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
```

Figure 17 — Terminal — application responding correctly

curl -I http://localhost:8080/dptweb/login → HTTP/1.1 401, Spring Security active

Quick-reference: common Phase 7 failures

Symptom	Check
User data still running	<code>sudo tail -f /var/log/cloud-init-output.log</code>
WAR not downloaded	IAM role attached correctly, and the bucket name matches exactly
App starts but throws DB errors	RDS endpoint/password in application.properties; rebuild and re-upload the WAR

PHASE 8 Application Load Balancer

Internet traffic should only ever reach the ALB — never the Tomcat instance directly.

8.1 Target group

Type	Instances
Name	javaauth-app-tg
Protocol / port	HTTP / 8080
VPC	javaauth-vpc
Health check protocol / path	HTTP / /dptweb/login
Health check success codes	200,401

⚠ Real issue hit at this step — Troubleshooting Log #8

With the default success code of 200 only, the target group kept marking a perfectly healthy instance as “Unhealthy.” The fix is a one-field change — see the log for why 401 is actually the correct healthy response here.

Register javaauth-app-1 and create the target group. Wait until the target turns healthy (green) before continuing.

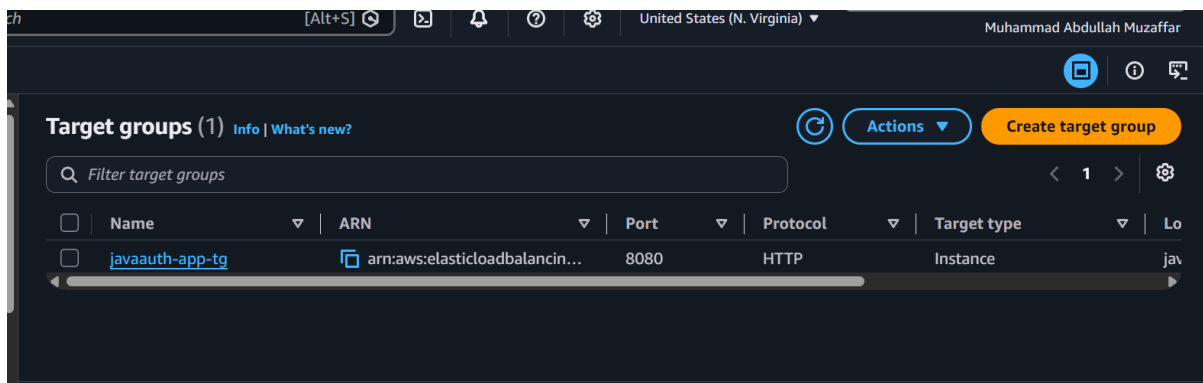


Figure 18 — EC2 console — target group created
javaauth-app-tg · HTTP : 8080 · Target type: Instance

8.2 The load balancer itself

Name	javaauth-alb
Scheme	Internet-facing
IP type	IPv4
VPC	javaauth-vpc
Mappings	javaauth-public-1a and javaauth-public-1b
Security group	javaauth-alb-sg only
Listener	HTTP :80 → javaauth-app-tg

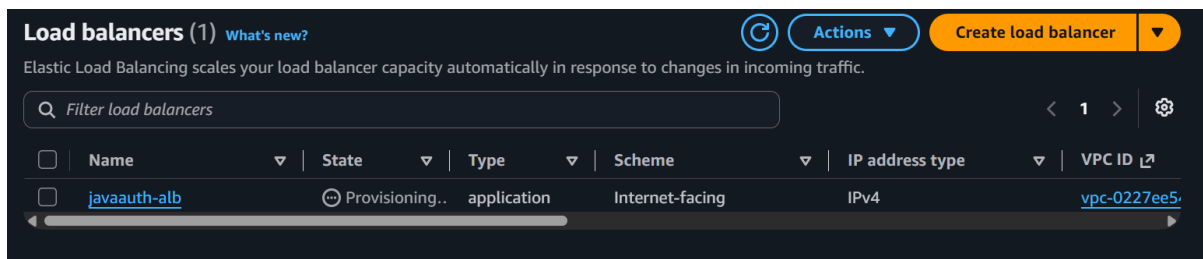


Figure 19 — EC2 console — load balancer coming online
javaauth-alb · Internet-facing · IPv4 · State: Provisioning

Create the ALB, copy its DNS name (e.g. javaauth-alb-xxxx.us-east-1.elb.amazonaws.com), and open it in a browser:

```
http://YOUR_ALB_DNS/dptweb/login
```

The Sign-in / Register screen should load. Final path confirmed end-to-end: Internet → ALB (public) → Tomcat (private) → RDS (private).

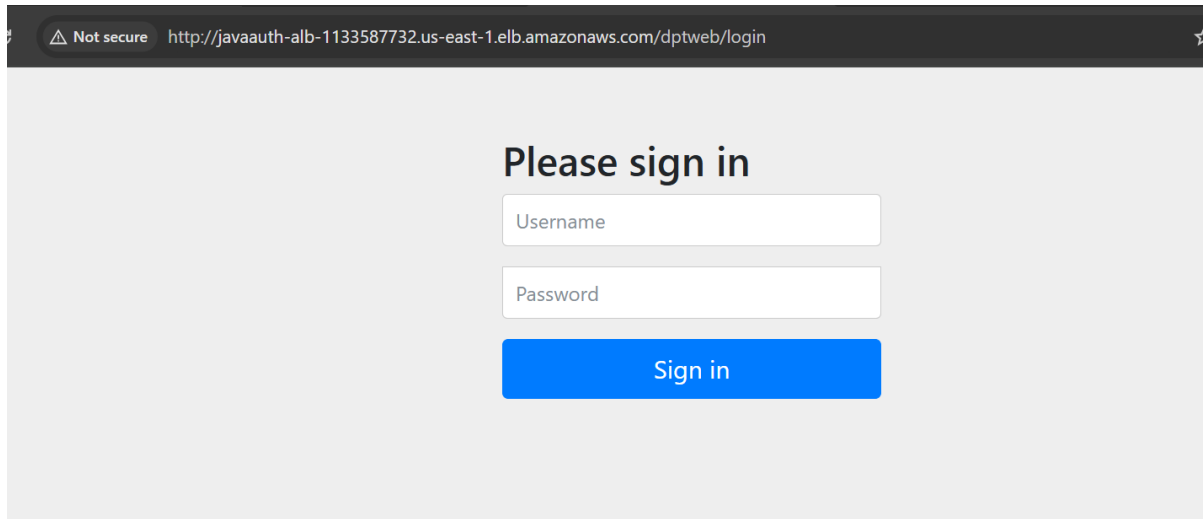


Figure 20 — Browser — the application, live on the internet
<http://javaauth-alb-1133587732.us-east-1.elb.amazonaws.com/dptweb/login>

PHASE 9 Launch Template and Auto Scaling

Do this only once the single manual instance is confirmed working behind the ALB.

9.1 Launch template

Name	javaauth-app-lt
AMI / type / key / IAM role / SG / user data	Same as javaauth-app-1 (Phase 7)
Network	No subnet baked in — the Auto Scaling Group chooses it
Auto-assign public IP	Disable

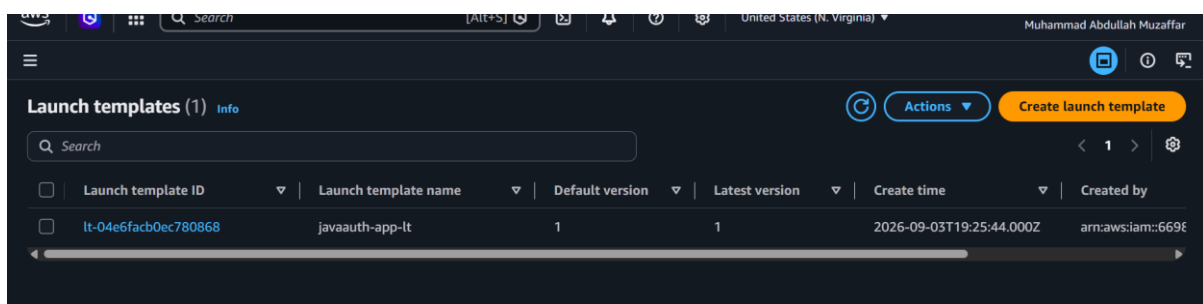


Figure 21 — EC2 console — launch template created
 javaauth-app-lt · lt-04e6facb0ec780868

9.2 Auto Scaling Group

Name	javaauth-app-asg
Launch template	javaauth-app-lt
VPC	javaauth-vpc
Subnets	javaauth-app-1a and javaauth-app-1b

Load balancing	Attach to existing target group javaauth-app-tg
Health check type	ELB
Health check grace period	300 seconds
Desired / Min / Max capacity	2 / 2 / 4
Scaling policy (optional)	Target tracking, average CPU 50%

Create the ASG, then terminate the original manually-launched javaauth-app-1 so it isn't paid for twice — the ASG will replace it automatically. Confirm two healthy targets, then refresh the ALB URL: the application tier is now multi-AZ.

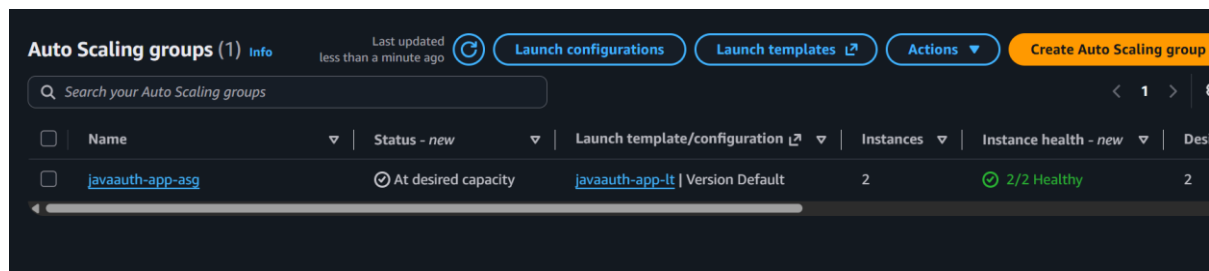


Figure 22 — EC2 console — Auto Scaling Group at target capacity
javaauth-app-asg · javaauth-app-lt · 2/2 instances healthy

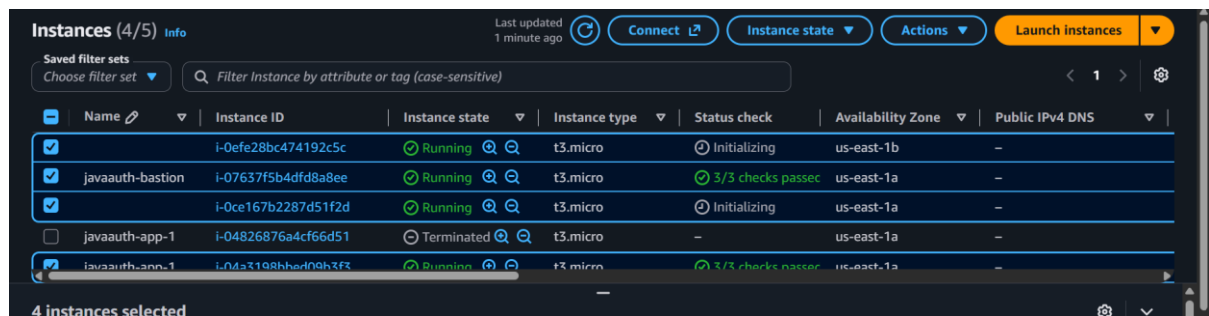


Figure 23 — EC2 console — scaling out across Availability Zones
New Auto Scaling instances launching while the original manually-created javaauth-app-1 shows Terminated

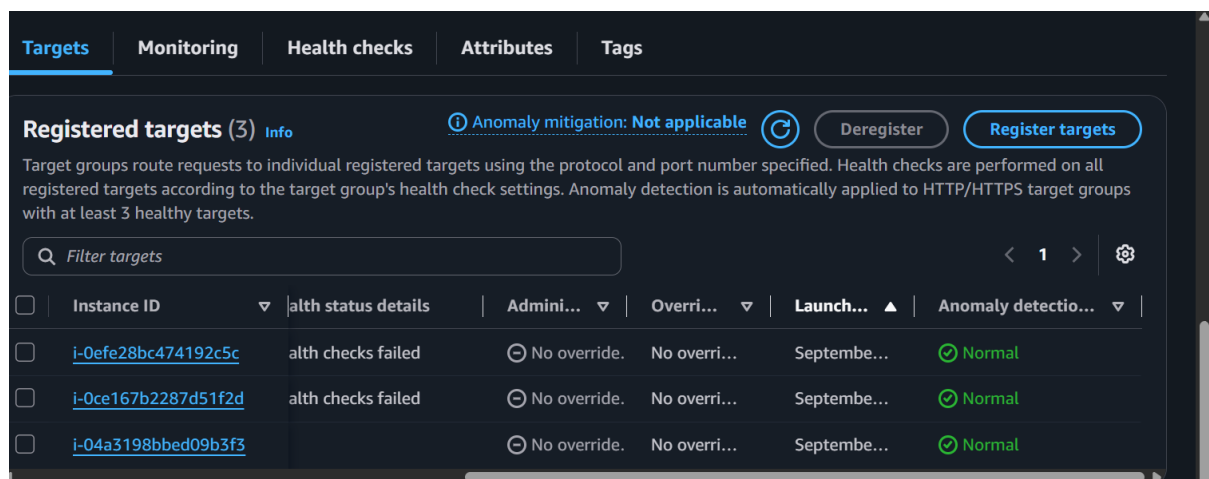


Figure 24 — EC2 console — new targets registering*The Auto Scaling Group's instances joining the target group alongside the original healthy target*

PHASE 10 Monitoring

10.1 CloudWatch alarms

CloudWatch → Alarms → Create alarm, for each of:

Metric	Suggested threshold
ALB UnHealthyHostCount	> 0 for 5 minutes
RDS CPUUtilization	> 80%
ASG CPUUtilization	> 70% (if used for a scale-out policy)

10.2 VPC Flow Logs (optional)

VPC → your VPC → Flow logs → send to CloudWatch Logs. Useful whenever something cannot connect and it isn't obvious why.

Recommended Next Steps

The base build above is deliberately minimal. Once it's confirmed working end-to-end, these are the natural next additions — in roughly the order they're worth tackling:

Feature	What it involves
HTTPS	ACM certificate → ALB listener 443 → redirect HTTP 80 to HTTPS (certificate and ALB must be in the same region)
Secrets Manager	Store the DB password there and fetch it at boot, instead of baking it into the WAR
Multi-AZ RDS	Modify DB → enable Multi-AZ, then test failover via Actions → Reboot with failover
CloudFront	Origin = ALB; adds HTTPS/caching, though this JSP-heavy app sees limited caching benefit
ElastiCache	Only worth it after adding session handling that actually uses Redis
Infrastructure as Code	Rebuild this same design in Terraform or CloudFormation to compare against the console build

Cleanup — De-provisioning Order

Delete resources in this exact order — dependencies matter, and skipping the order leaves orphaned resources that keep billing quietly in the background.

#	Resource	Note
1	Auto Scaling Group	Terminates the app instances automatically
2	Launch Template	—
3	Load Balancer	—
4	Target Group	—
5	Bastion EC2	Terminate

#	Resource	Note
6	RDS instance	Skip final snapshot for a lab environment
7	DB Subnet Group	—
8	NAT Gateway	Wait for full deletion before continuing
9	Elastic IP	Release once unattached
10	S3 objects + bucket	Empty the bucket first, then delete it
11	VPC	Deletes associated subnets, route tables, and gateways together
12	Key pair, IAM role, remaining security groups	—
13	CloudWatch log groups / alarms	—

Forgotten Elastic IPs and NAT Gateways are the most common source of surprise charges — check EC2 → Elastic IPs is empty as a final step.

Appendix — Troubleshooting & Error Resolution Log

Every error below was hit for real during this build. Each entry gives the exact symptom, the actual root cause, and the fix that was applied — plus a permanent fix where one exists, so it doesn't happen again on the next build.

Quick index

#	Issue	Phase	Category
1	Maven can't resolve the Spring Boot parent POM	5 — Build	Local network
2	Maven: mysql-connector-java version missing	5 — Build	Dependency management
3	IAM console “Deserialization error”	6 — IAM Role	AWS account state
4	NAT Gateway in the wrong subnet	1 — VPC / 7 — App server	Networking
5	User data script crashed on a stray space	7 — App server	Deployment script
6	SSH ProxyJump denied from Windows	7 — App server	SSH / access
7	Duplicate org_apache_jasper_el fragment (HTTP 404)	7 — App server	WAR packaging
8	ALB marked a healthy instance “Unhealthy”	8 — Load Balancer	Health checks

#1

Maven couldn't resolve the Spring Boot parent POM

Local network • Encountered in Phase 5 (Build the WAR)

Symptom

```
[FATAL] Non-resolvable parent POM for com.devopsrealtime:dptweb:1.0:
Could not transfer artifact org.springframework.boot:spring-boot-starter-parent:pom:2.7.18
from/to central: No such host is known (repo.maven.apache.org)
```

Root cause

The local machine had no working DNS/internet route to Maven Central at the moment the build ran — this was a connectivity problem, not a project configuration problem.

Fix applied

Confirmed general internet connectivity, then re-ran the build. Once the network was back, Maven resolved and downloaded the parent POM and every dependency normally.

```
mvn clean package -DskipTests
```

Permanent fix / how to avoid it next time

If this happens behind a corporate proxy, add a <proxies> block to settings.xml. If a partial download left corrupt cache entries, add -U to force fresh dependency resolution: `mvn clean package -DskipTests -U`

#2

Maven: missing version for the MySQL connector

Dependency management • Encountered in Phase 5 (Build the WAR)

Symptom

```
[ERROR] 'dependencies.dependency.version' for mysql:mysql-connector-java:jar is missing.
@ line 42, column 15
```

Root cause

pom.xml declared the legacy mysql:mysql-connector-java artifact with no explicit <version>, relying on Spring Boot's dependency-management BOM to supply one. The project's Maven mirror (a private JFrog Artifactory instance) doesn't carry that legacy version mapping, so resolution failed.

Fix applied

Replaced the dependency with the modern coordinates that Spring Boot 2.7.18 manages directly:

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

Permanent fix / how to avoid it next time

Alternative if the legacy artifact must be kept: pin an explicit version instead, e.g. <version>8.0.33</version>.

#3

IAM console “Deserialization error” creating the EC2 role

AWS account state • Encountered in Phase 6 (IAM Role)

Symptom

```
Unexpected error
network error Deserialization error: to see the raw response,
inspect the hidden field {error}.$response on this object.
```

Root cause

The AWS account was still inside its 48-hour new-account verification hold, which also blocked AWS CloudShell (“Your account verification is in progress”). The console couldn't load the full managed-policy list from the backend, so the standard Roles → Create role wizard kept failing on the permissions step.

Fix applied

Used the inline “Create new IAM profile” shortcut available directly from the EC2 launch screen's IAM instance profile field. That shortcut skips the broken policy-search UI and accepts a custom JSON permissions statement pasted directly — granting s3:GetObject / s3:ListBucket plus the CloudWatch logging actions needed for this build.

Permanent fix / how to avoid it next time

Once the account finished its verification window, the standard Roles → Create role wizard worked normally with no further issues.

#4

NAT Gateway placed in a private subnet

Networking • Encountered in Phase 1 (root cause) / Phase 7 (symptom)

Symptom

```
cloud-init-output.log:
Curl error (28): Timeout was reached for
https://a12023-repos-us-east-1-....s3.amazonaws.com/... [Connection timed out]

sudo systemctl status tomcat → Unit tomcat.service could not be found.
```

Root cause

The NAT Gateway had been created inside javaauth-app-1b, a private subnet, instead of a public one. A NAT Gateway only functions with a route to an Internet Gateway, and Internet Gateways only exist in public subnets — so the private app tier had no path out to the internet, and “dnf update” timed out before Tomcat could even be installed.

Fix applied

Deleted the NAT Gateway and recreated it inside javaauth-public-1a with a fresh Elastic IP. Confirmed the private route table's 0.0.0.0/0 route still pointed at the new gateway's ID, then terminated the stuck app instance and launched a fresh javaauth-app-1.

Permanent fix / how to avoid it next time

The EC2 user data script only runs on first boot — an instance that failed partway through will not retry on its own. Once the network is fixed, replace the instance rather than waiting on it.

#5

User data script crashed on a stray space in the S3 path

Deployment script • Encountered in Phase 7 (App server)

Symptom

Even after the NAT Gateway fix:
`sudo systemctl status tomcat` → Unit tomcat.service could not be found.

Root cause

The script starts with `set -e` (abort on first error). The S3 download line had an accidental space: “aws s3 cp s3://javaauth-artifacts-abdullah/dptweb-1.0.war ...” — the AWS CLI couldn't parse that path, the command failed, and `set -e` killed the rest of the script before tomcat.service was ever created.

Fix applied

Removed the space (s3://javaauth-artifacts-abdullah/...), terminated the broken instance, and launched a new one with the corrected script. Tomcat installed and started cleanly on the next boot.

#6

SSH ProxyJump (-J) denied from Windows PowerShell

SSH / access • Encountered in Phase 7 (SSH via bastion)

Symptom

```
ssh -i "javaauth-key.pem" -J ec2-user@BASTION_IP ec2-user@APP_PRIVATE_IP
ec2-user@BASTION_IP: Permission denied (publickey,gssapi-keyex,gssapi-with-mic)
Connection closed by UNKNOWN port 65535
```

Root cause

The key itself, its Windows file permissions, and the bastion's key pair were all correct — a direct single-hop connection to the bastion worked fine every time. Only the two-hop -J jump failed, most likely a Windows OpenSSH quirk in how it forwards the identity file across the proxy hop.

Fix applied

Connected to the bastion normally, copied the private key onto the bastion itself, locked down its permissions, then ran the second hop from inside the bastion instead of from Windows directly:

```
# on the bastion
nano javaauth-key.pem          # paste the key contents, save
```

```
chmod 400 javaauth-key.pem
ssh -i "javaauth-key.pem" ec2-user@APP_PRIVATE_IP
```

Permanent fix / how to avoid it next time

Copying a private key onto a bastion host is an acceptable lab shortcut but not a production practice — a production setup should use ssh-agent forwarding (ssh -A) instead, so the key never leaves the local machine.

#7

Duplicate org_apache_jasper_el fragment — HTTP 404

WAR packaging • Encountered in Phase 7 (verification) — root cause in Phase 5 (pom.xml)

Symptom

```
curl -I http://localhost:8080/dptweb/ → HTTP/1.1 404

catalina.out:
java.lang.IllegalArgumentException: More than one fragment with the name
[org_apache_jasper_el] was found. Duplicate fragments found in
[.../tomcat-embed-el-9.0.83.jar, .../tomcat-jasper-el-9.0.31.jar]
```

Root cause

pom.xml explicitly declared org.apache.tomcat:tomcat-jasper:9.0.31 with no <scope>, so Maven packaged it — and its embedded EL library — straight into dptweb-1.0.war. Once deployed, that bundled library collided with the same EL classes already shipped inside the standalone Tomcat 9.0.98 install, and Tomcat refused to start the web application context.

Fix applied

Deleted the duplicate jars directly from the unzipped application folder and restarted Tomcat:

```
sudo rm -f /opt/tomcat/webapps/dptweb/WEB-INF/lib/tomcat-embed-el-9.0.83.jar
sudo rm -f /opt/tomcat/webapps/dptweb/WEB-INF/lib/tomcat-jasper-el-9.0.31.jar
sudo systemctl restart tomcat
```

The app came up immediately afterward, returning HTTP 401 — confirming Tomcat and Spring Security were both now active.

Permanent fix / how to avoid it next time

Mark the dependency provided in pom.xml so it is never bundled into the WAR in the first place:

```
<dependency>
  <groupId>org.apache.tomcat</groupId>
  <artifactId>tomcat-jasper</artifactId>
  <version>9.0.31</version>
  <scope>provided</scope>
</dependency>
```

#8

ALB marked a healthy instance “Unhealthy”

Health checks • Encountered in Phase 8 (Target Group)

Symptom

```
Target group health status: Unhealthy
(despite curl -I http://localhost:8080/dptweb/login returning a real response on the server
itself)
```


Root cause

/dptweb/login is protected by Spring Security, so an unauthenticated request — like the ALB's own health check — correctly receives 401 Unauthorized, not 200 OK. The app was genuinely healthy; the target group's default success-code range just didn't accept 401.

Fix applied

Changed the target group's health check success codes from 200 to 200,401. The target flipped to Healthy on the very next check cycle.

End of document.